PHASE 5 OF 8

Project Development

Building the backend, frontend and document generators

Project: LegalEase – AI-Powered Legal Document Generator

Presented by: Ganesh Prabhu, Gopinath, Hariharan, Gowtham

5.1 Purpose of this Phase

The Project Development phase is where the design was turned into a working application. The team built the backend API, the Streamlit frontend and the three document-generation modules, and then connected them into a single workflow. Development followed the order set in the plan: backend first, then frontend, then formatting and integration.

5.2 Development Environment Setup

1. Install Python and create a virtual environment for the project.
2. Install the dependencies: Streamlit, FastAPI, Uvicorn, the Google Gemini SDK, python-docx, FPDF, Pillow, Requests and python-dotenv.
3. Create an environment (.env) file that stores the Gemini API key.
4. Confirm that both the backend server and the Streamlit app start without errors.

```
pip install streamlit fastapi uvicorn google-generativeai
pip install python-docx fpdf pillow requests python-dotenv
```

5.3 Backend Development

The backend is a FastAPI application that exposes the document-generation service. The main development activities were:

- **Configure FastAPI** and run it with Uvicorn.
- **Create the /generate POST endpoint** that receives the user's inputs.
- **Validate requests using Pydantic** so missing or incorrectly typed fields are rejected before any AI call is made.
- **Connect the backend to the Gemini document generator**, which builds the prompt from the inputs, calls Gemini and returns the generated text.
- **Handle errors** so that AI or network problems produce a readable message.

5.4 Frontend Development

The frontend is built with Streamlit, which allows a working interface to be created in Python without separate web-design code.

- Build the interface layout with a title and short instructions.

- Add inputs for document type, parties, terms, effective date and related details such as jurisdiction.
- Add a *Generate Document* button that sends the inputs to the backend using the Requests library.
- Display the generated content in a preview area.
- Allow the user to edit the content before export.
- Add download buttons for TXT, DOCX and PDF.

5.5 Document Generation Modules

Format	Implementation	Features
TXT	Plain text output of the final content	Simple, universally readable
DOCX	python-docx builds a formatted Word document	Headings, paragraphs for terms, optional logo and footer
PDF	FPDF builds a formatted PDF document	Headings, terms, logo and footer; Pillow assists with images

Formatted outputs can include headings, terms, a logo and footer information. Each formatter takes the final (edited) text as input, so the downloaded file always reflects what the user sees in the preview.

5.6 Integration

After each module worked on its own, the team joined them end to end: the Streamlit form calls FastAPI, FastAPI calls Gemini, the result returns to the preview, and the export buttons call the formatters. Integration also involved matching field names between the form and the Pydantic model and making sure both servers could run at the same time.

```
Example request to /generate (illustrative)
{
  "document_type": "Lease Agreement",
  "parties": "Party A and Party B",
  "terms": "Duration, rent and obligations",
  "effective_date": "2026-01-01",
  "jurisdiction": "Tamil Nadu, India"
}
```

The exact field names above are illustrative; the implementation uses the fields defined in the project's Pydantic request model.

5.7 Running the Application

```
# Terminal 1 - backend
uvicorn main:app --reload
```

```
# Terminal 2 - frontend
streamlit run app.py
```

File names such as main.py and app.py are examples; use the names of the actual project files.

5.8 Challenges and Solutions

Challenge	Approach
Keeping generated text well structured	Refine the prompt to request title, parties, clauses and signature section
Different appearance in DOCX and PDF	Use one shared sample input and adjust each formatter until both look consistent
Running two servers together	Document the start-up order and required ports
Protecting the API key	Load it from the .env file instead of writing it in code

5.9 Outcome of the Phase

At the end of development LegalEase could accept user input, generate a legal-document draft with Gemini, display and edit it, and export it in three formats. The application was then handed to the Testing phase for verification.

5.10 Component Summary

Component	Technology	Development result
Frontend	Streamlit	Form, preview, edit area and download buttons
Backend	FastAPI and Uvicorn	/generate endpoint with Pydantic validation
AI Core	Google Gemini	Prompt building and document text generation
Formatters	python-docx, FPDF, Pillow	TXT, DOCX and PDF output with logo and footer
Configuration	python-dotenv	API key and settings loaded from .env

5.11 Development Practices Followed

- Each module was built and checked on its own before integration.
- Responsibilities were kept separate: interface code does not contain AI logic, and formatters do not call the API.

- The API key was never written into source code.
- Errors from Gemini or the network are reported to the user in plain language.
- Small, frequent runs of the whole application were used to catch integration problems early.

5.12 Handover to Testing

The completed application, its setup steps and the list of features (input, generation, preview, edit, three exports) were given to the testing activity so that each feature could be checked against the requirements.